Activity 2 – CoT

Scenario: Customer tries to transfer ₹50,000. Transaction failed. Error shown: "Transaction could not be completed." Act as QA to understand what could be all the possible reasons for the transaction to be fail and let me know the CoT when you do asses the scenario

Recalled 2 memories
Recalled 2 memories

Recalled 2 memoriesQa Skills, Recent Work

Systematized payment failure root causes and investigation methodology.
Done
Here's how I'd break this down as a QA — first the possible root causes, then the actual chain-of-thought investigation process.

Possible Root Causes (by category)

1. Funds / Account State

- Insufficient balance at the exact moment of debit (race condition vs. displayed balance)
- Source account frozen, dormant, or under a hold (lien, legal freeze, RBI/compliance flag)
- KYC expired or re-KYC pending, blocking large-value transactions

2. Limits & Thresholds

- Per-transaction limit exceeded (e.g., UPI apps often cap single transactions at ₹1,00,000, but some banks/PSPs set lower default caps like ₹25,000–₹50,000 unless raised)
- Daily cumulative limit exceeded when combined with earlier transactions same day
- New beneficiary cooling-period limit (many banks cap first 24 hrs to new payees at ₹25,000–₹50,000 — ₹50,000 is a classic edge-case value for this)

3. Beneficiary Related

- Invalid/inactive IFSC or beneficiary account closed
- Beneficiary not yet "activated" (some banks need beneficiary addition + 30 min–24 hr wait)
- Beneficiary bank's core banking system down (receiving side failure surfacing as generic error on sender side)

4. Authentication

- OTP timeout, OTP mismatch, or SMS delivery delay
- Biometric/UPI PIN failure exceeding retry threshold
- Session expiry mid-transaction (token expired between initiation and confirmation)

5. Network / Infrastructure

- Timeout between bank server and NPCI switch (UPI) or IMPS/NEFT gateway
- Payment gateway or middleware API timeout/5xx error
- Core banking system (CBS) unavailable or under maintenance window
- Load balancer / connection drop mid-transaction (this is the dangerous one — debit may have occurred, credit didn't)

6. Timing / Scheduling

- NEFT batch cut-off time missed (transaction queued, generic failure shown instead of "queued")
- Bank-side scheduled maintenance window
- Bank holiday affecting NEFT/RTGS specifically (UPI/IMPS unaffected)

7. Risk & Compliance

- Fraud detection engine auto-blocking (velocity checks, unusual amount/pattern, first-time large transfer)
- AML/sanctions screening flag on beneficiary

8. Client/App Layer

- Stale app version with a known bug in the payment flow
- Corrupted local cache/session data
- Duplicate transaction detection blocking a retry that looks identical

9. Error Handling Itself (a QA finding, not just a cause)

- The generic message "Transaction could not be completed" is itself a defect symptom — it likely maps 10+ backend error codes to one string, hiding whether it's a client error, server error, timeout, or business-rule rejection. That's something I'd flag regardless of root cause.

Chain of Thought — how I'd actually investigate this

Step 1 — Establish reproducibility and blast radius

Is this a one-off or consistent? Same user only, or multiple users? Same beneficiary, or any beneficiary? This immediately splits the investigation into "environmental/data issue" vs. "systemic bug."

Step 2 — Check the most critical thing first: did money move?

Before anything else, verify via backend/DB logs whether the debit actually happened on the source account even though the UI shows failure. This is the highest-severity possibility (debited but not credited) and determines if this is a P1/data-integrity bug vs. a UX/validation bug.

Step 3 — Pull the actual backend error code

The UI message is generic by design or by defect — I go straight to API/gateway logs to get the real error code (timeout, limit breach, auth failure, downstream 5xx, etc.). This single step usually eliminates 70% of the hypothesis list.

Step 4 — Isolate the layer

Client (app/browser) → API gateway → core banking → NPCI/IMPS/NEFT switch → beneficiary bank. I trace the transaction ID through each hop to find exactly where it stopped or errored.

Step 5 — Correlate against known constraints

Check: transaction limits configured for this user/channel, beneficiary age (new vs. existing payee), time of transaction against NEFT batch windows, and whether ₹50,000 specifically crosses a configured threshold (this amount is a common trigger value, so I test that hypothesis early).

Step 6 — Check environment/test data validity

If this is in a test environment: is the test account funded correctly, is the beneficiary properly onboarded, are sandbox limits different from prod, is the UPI/IMPS simulator behaving as expected?

Step 7 — Attempt controlled reproduction with variations

Same amount, different beneficiary. Same beneficiary, lower amount (e.g., ₹10,000). Different channel if available (UPI vs IMPS vs NEFT). Different time of day. This isolates whether it's amount-specific, beneficiary-specific, or channel-specific.

Step 8 — Check timing correlation with infra events

Cross-reference the failure timestamp against any deployment, maintenance window, or downstream outage (NPCI/bank-side) logs.

Step 9 — Classify and log

Once root cause is confirmed, I log it with: exact error code, transaction ID, layer where it failed, whether funds moved, and reproducibility — plus a separate defect for the vague UI messaging itself, since that alone increases support tickets and makes triage harder for both QA and customers.

Want me to turn this into a structured test case / defect template, or a decision-tree diagram for triaging this class of failure?